

2 – Performance evaluation

Machine Learning (MPA-MLR)

Tomáš Vičar

2025

Department of Biomedical Engineering,
FEEC, Brno University of Technology

Outline

Supervised pipeline

Performance evaluation definition

Evaluation metrics

Class Imbalance

Validation approaches

Preprocessing

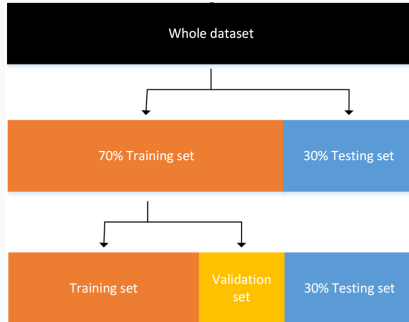
Supervised pipeline

Supervised Learning Pipeline

1. **Define goal** — what do we want to predict?
2. **Split data** — training / validation / test.
3. **Preprocess** — clean and prepare features.
4. **Train & tune** — fit models, adjust settings.
5. **Evaluate** — check performance on validation.
6. **Final test** — use test set once for real estimate.

Performance evaluation definition

Dataset division



- **Training set** – for model training
- **Validation set** – used to tune hyperparameters, compare models, and prevent overfitting
- **Test set** – kept aside until the end, provides an unbiased estimate of generalization performance

Supervised Learning Goal

Ultimate goal: Not to minimize training error, but to find a model that performs well on *unseen data*.

Population risk (ideal but unknown):

$$R(\theta) = \mathbb{E}_{(x,y) \sim P} [\mathcal{L}(f_{\theta}(x), y)]$$

P = true (unknown) data distribution; $R(\theta)$ = expected loss on future samples; $\mathcal{L}$ = some evaluation metric.

Practical approximation:

- We train by minimizing the *empirical risk* on training data

$$\hat{R}_{\text{train}}(\theta) = \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} \mathcal{L}(f_{\theta}(x_i), y_i).$$

- But what we care about is the expected risk $R(\theta)$.
- Therefore we evaluate on a separate **test set**

$$\hat{R}_{\text{test}}(\theta) = \frac{1}{n_{\text{test}}} \sum_{j=1}^{n_{\text{test}}} \mathcal{L}(f_{\theta}(x_j), y_j) \approx R(\theta).$$

Evaluating on a Test Set

Key idea: The test set simulates *new, unseen samples* that the classifier will face in practice.

- During training, the model sees only the **training set**.
- If we also measure performance on the training set, we may get an **overly optimistic** result.
- By setting aside a **test set** (not used in training), we get a realistic estimate of how well the model generalizes.
- The test set should be drawn from the same distribution as future data.
- Test performance $\approx$ expected performance on future unseen samples.

Why evaluate performance?

- Get a reliable estimate of the classifier performance – i.e., how it will perform on new – so far unseen – data instances. .
- Compare your classifiers that you have developed – to decide which one is “the best” .

Model Development as Optimization Problem

Ultimate goal: Find the *whole pipeline* (features, preprocessing, model, hyperparameters) that minimizes prediction error on unseen data.

Abstract formulation:

$$\mathcal{P}^* = \arg \min_{\mathcal{P} \in \mathcal{H}} \hat{R}_{\text{val/test}}(\mathcal{P})$$

- $\mathcal{P}$ = complete pipeline (feature extraction, preprocessing, model, hyperparameters) - $\mathcal{H}$ = set of all candidate pipelines - $\hat{R}_{\text{val/test}}(\mathcal{P})$ = estimated risk on a held-out validation/test set

Whole model development = pipeline optimization for lowest test error.

Two Optimization Problems in Supervised Learning

But in most cases, training alone is also optimization problem, so we have: **1.**

Training optimization (inner problem):

$$\theta^* = \arg \min_{\theta} \hat{R}_{\text{train}}(\theta) = \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} \mathcal{L}(f_{\theta}(x_i), y_i).$$

- Adjust model parameters θ to fit the training data. - This is what learning algorithms (SGD, backpropagation, etc.) actually solve.

2. Model development optimization (outer problem):

$$\mathcal{P}^* = \arg \min_{\mathcal{P} \in \mathcal{H}} \hat{R}_{\text{val/test}}(\mathcal{P}).$$

- $\mathcal{P}$ = whole pipeline (features, preprocessing, model class, hyperparameters). - Goal = find pipeline with best expected performance on unseen data.

Key point: Training solves an *inner optimization*, but our true goal is the *outer optimization* of generalization.

Role of the Validation Set

Problem: If we tune hyperparameters or select features using the test set, we “cheat” and performance will be too optimistic.

Solution: Introduce a **validation set**:

- Separate from training data.
- Used to tune hyperparameters, select features, compare models.
- Prevents **overfitting to the test set**.

Workflow:

- Train → Validate → Tune → (repeat)
- After final model is chosen, use the **test set only once** for unbiased evaluation.

We Need a Measure of Performance

Why? Optimization means “searching for the best” — but we must first define what **best** means.

Loss / Cost function:

- A numerical score that tells us how bad a prediction is.
- Small value = good prediction, large value = bad prediction (or vice versa).
- Guides the learning algorithm during training or evaluates the final model performance

Takeaway: Without a loss function, the computer would not know which model is better.

Let's define some metrics for that...

Evaluation metrics

Basic Evaluation Metrics

For classification:

- **Accuracy** – proportion of correctly classified samples

$$\text{Accuracy} = \frac{\#\{\hat{y}_i = y_i\}}{n} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Error rate** – proportion of misclassified samples: $\text{Error} = 1 - \text{Accuracy}$

For regression:

- **Mean Squared Error (MSE) and Mean Absolute Error (MAE)**

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 \quad \text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

- **Root Mean Squared Error (RMSE)** – square root of MSE with interpretable in units of y .

code example

Confusion Matrix (Binary) based metrics

		Predicted Class		
		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Sensitivity / Recall $\frac{TP}{(TP + FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specificity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predicted Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + TN + FP + FN)}$

Sensitivity, specificity, PPV, NPV are not sufficient alone - model that predict only positive class has sensitivity 1.

$$F_1 \text{ (Dice)} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN} = \frac{2|X \cap Y|}{|X| + |Y|}$$

Geometric mean of sensitivity and specificity. Useful in cases with a small fraction of positive samples.

ROC: Intuition

Setting: A classifier outputs a *score* (confidence) (e.g. logistic regression). We pick a *threshold* to decide Positive/Negative. Sometimes the threshold is part of the model (e.g SVM).

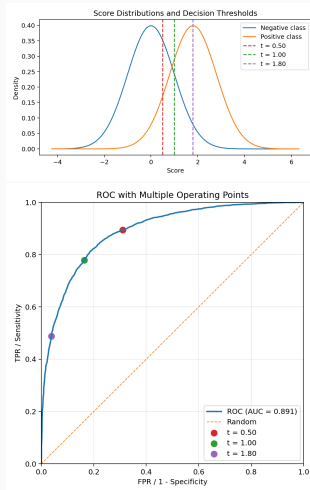
ROC curve shows trade off:

$$\text{TPR (Recall, Sensitivity)} = \frac{TP}{TP + FN}$$

$$\text{FPR (1 - Specificity)} = \frac{FP}{FP + TN}$$

as we sweep the threshold.

- Each threshold $\Rightarrow$ one point (FPR, TPR).
- The **ROC curve** connects these points.
- **Perfect** classifier touches (0, 1).
- **Random** classifier lies on the diagonal.
- More information than just one value.



Constructing the ROC Curve and AUC

Procedure

1. Compute scores s_i for all samples.
2. Sort unique score values and use them as thresholds.
3. For each threshold, binarize predictions and compute TPR, FPR.
4. Plot (FPR, TPR) and connect. `code example`

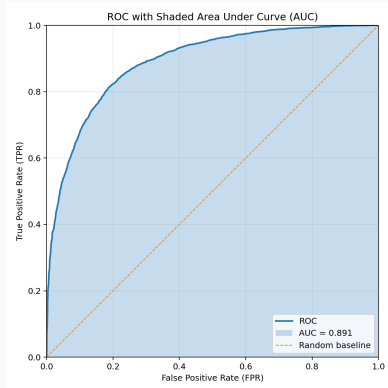
ROC AUC (Area Under Curve)

$$\text{AUC} = \int_0^1 \text{TPR}(\text{FPR}) d(\text{FPR})$$

Interpretation:

$$\text{AUC} = \Pr(s(x^+) > s(x^-))$$

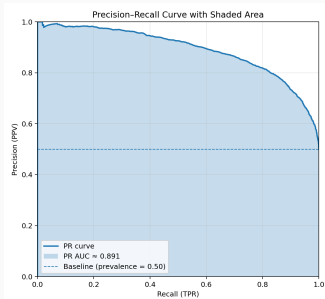
AUC = probability that a randomly chosen positive has a higher score than a randomly chosen negative.



Precision–Recall (PR) Curve

Idea: Same as ROC — vary the threshold and plot a trade-off. But here the trade-off is between **precision** and **recall**.

- Random baseline = the prevalence of positives in the data.
- PR AUC close to 1 = excellent separation; close to baseline = poor.
- Useful for **imbalanced datasets** where positives are rare (e.g., disease detection, fraud, anomalies).
- More informative than ROC for imbalanced — ROC may look “too good”. `code example`



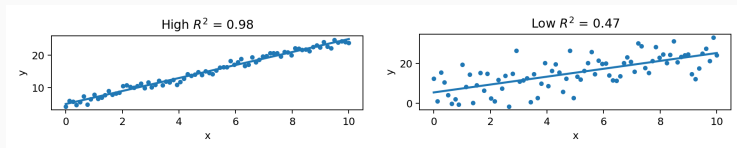
R^2 (Coefficient of Determination)

Idea: How much better the model is than always predicting the mean.

Definition

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

- Numerator = Sum of Squares Error (residual) (SSE)
- Denominator = Sum of Squares Total (SST)
- $R^2 \in (-\infty, 1]$; closer to 1 means better fit. Prediction of mean $\rightarrow 0$.



code example

Multiclass vs. Multilabel

Multiclass: exactly one label out of K classes (single-label) - **only one can be true**

$$y \in \{1, \dots, K\} \quad \text{or one-hot} \quad y \in \{0, 1\}^K \quad \text{with} \quad \sum_{k=1}^K y_k = 1$$

(one-hot example: $y = [0, 1, 0, 0, 0]$)

Examples:

- Brain tumor type on MRI: {Meningioma, Glioma, Pituitary}
- Diabetic retinopathy severity: {No, Mild, Moderate, Severe, PDR}

Multilabel: zero, one, or many labels per sample (multi-hot - **several can be true**)

$$y \in \{0, 1\}^K \quad (\text{e.g., } y = [1, 0, 0, 1, 1])$$

Examples:

- Image tags: {cat, outdoor, night}
- Patient comorbidities: multiple diseases can be present simultaneously

Basic Metrics: Multiclass vs. Multilabel

We can aggregate metrics for individual classes/labels:

Averaging schemes (for multi-class / multi-label)

- **Micro:** pool all decisions across classes/labels first (TP, FN...), then compute one global (reflects frequent classes more).
- **Macro:** compute per-class/label metric, then average (treats all classes equally).
- **Weighted macro** uses class supports (count).

Multiclass (single label)

- **Accuracy** — overall fraction correct
- **Macro-F1** — average F1 across classes (handles imbalance)

Multilabel (multi-hot)

- **Subset accuracy (exact match)** — all labels must be correct
- **Hamming loss** — per-label error rate (lower is better)
- **Micro/Macro F1** — aggregate across labels

Macro vs Weighted vs Micro

		Predicted		
		Airplane	Boat	Car
Actual	Airplane	2	1	0
	Boat	0	1	0
	Car	1	2	3

Macro-average

- Treat all classes equally — rare classes matter.
- Use with class imbalance when you want fairness across classes.
- Highlights poor performance on minority classes (but can be noisy for tiny classes).

Weighted macro-average

- Like macro, but weighted by class support (prevalence).
- Use when you want class fairness *and* to reflect real-world frequencies.
- More stable than plain macro when some classes are very small (good default under imbalance).

Micro-average

- Pools all decisions — measures overall instance-level performance.
- Use when overall volume matters (may be dominated by majority classes).
- In single-label multiclass, micro-F1 equals accuracy.

code example

Label	True Positive (TP)	False Positive (FP)	False Negative (FN)	Precision	Recall	F1 Score
Airplane	2	1	1	0.67	0.67	$2 * (0.67 * 0.67) / (0.67 + 0.67) = 0.67$
Boat	1	3	0	0.25	1.00	$2 * (0.25 * 1.00) / (0.25 + 1.00) = 0.40$
Car	3	0	3	1.00	0.50	$2 * (1.00 * 0.50) / (1.00 + 0.50) = 0.67$

Label	Per-Class F1 Score	Macro-Averaged F1 Score
Airplane	0.67	$\frac{0.67 + 0.40 + 0.67}{3} = 0.58$
Boat	0.40	
Car	0.67	

Label	Per-Class F1 Score	Support	Support Proportion	Weighted Average F1 Score
Airplane	0.67	3	0.3	$(0.67 * 0.3) + (0.40 * 0.1) + (0.67 * 0.6) = 0.64$
Boat	0.40	1	0.1	
Car	0.67	6	0.6	
Total	-	10	1.0	

Label	True Positive (TP)	False Positive (FP)	False Negative (FN)	Micro-Averaged F1 Score
Airplane	2	1	1	$\frac{TP}{TP + \frac{1}{2}(FP + FN)} = \frac{6}{6 + \frac{1}{2}(4 + 4)} = 0.60$
Boat	1	3	0	
Car	3	0	3	
TOTAL	6	4	4	

Class Imbalance

Unbalanced Datasets (Class Imbalance)

- One class is much rarer than others (e.g., positives $< 5\%$).
- **Accuracy paradox:** high accuracy even if minority is ignored.

Evaluate properly

- Use **macro / weighted F1**.
- Show **normalized confusion matrix** and report prevalence.

Training options

- **Class weights / cost-sensitive loss**.
- **Resampling:** undersample majority, oversample minority.

Handling Imbalance: Oversampling vs. Undersampling

Oversampling (increase minority)

- *Random oversampling*: duplicate minority examples; most popular
- Can use methods to synthesize new minority points (SMOTE, ADASYN).
- **Pros**: keeps all majority data; boosts recall
- **Cons**: risk of overfitting (dupes) or artifacts (synthetic)
- **Use when**: very small minority

Undersampling (reduce majority)

- *Random undersampling*: drop majority examples; most popular
- Can use complex cleaning rules: Tomek links, ENN, NearMiss, ClusterCentroids
- **Pros**: faster; can remove borderline noise/overlap
- **Cons**: discards information; possible underfitting
- **Use when**: huge majority class; training cost high

Best Practices (Sampling & Splits)

Pipeline & leakage

- Apply any **sampling only on the training data, inside each CV fold**; never on validation/test.
- Do preprocessing (scaling, PCA) *inside the split* too, fitted on training only. Values from training data are used for validation/test..

Stratify splits (important)

- Keep class proportions similar in train/val/test and in each fold - stratified.
- With groups (patients/devices/sites): use all data from each group in train/validation/test – > groups don't leak across splits.
E.g., when multiple signals come from the same patient, assign all of that patient's signals to a single split. Otherwise you get leakage and an overoptimistic result, because signals from the same patient are highly correlated.

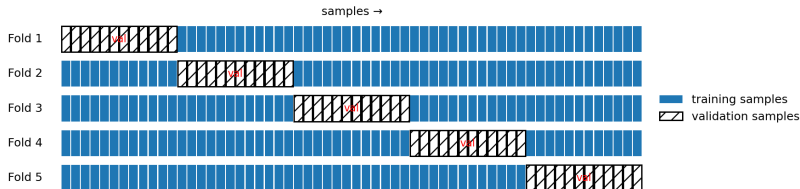
Validation approaches

Cross-Validation (k-fold)

- Split data into k (roughly) equal folds; repeat k rounds.
- Each round: **train** on $k-1$ folds, **validate** on the remaining fold.
- Report the mean $\pm$ std of the validation metric across folds.
- Use **Stratified** (and **Group-aware**) CV when classes or groups exist.
- Keep the **test set untouched**; scaling/sampling happen *inside each fold*.
- If $k = n$ (number of samples), it is **leave-one-out** CV.
- With small datasets can be used also for testing.

$\uparrow k \Rightarrow \uparrow$ computation (more fits)

$\downarrow k \Rightarrow \downarrow$ training data per fit ($n \frac{k-1}{k}$)



code example

Hold-out Validation (Single Split)

- Split data *once* into **train** / **validation** / **test** (e.g. 70/15/15 or 60/20/20).
- Train on **train**; tune hyperparameters/thresholds on **validation**; report final performance on **test (once)**.

Pros

- Simple and fast (only one fit per model/setting).
- Works well when the dataset is large.
- Easy to keep a clean, untouched test set.

Cons

- Higher variance estimate — results depend on the particular split.
- With limited data, you may waste samples that could help training.

code example

Preprocessing

Preprocessing

Why it matters

- Models assume comparable feature scales, no NaNs, and sensible encodings.
- **Preprocessing must be trained only on training data** and applied to val/test.

Typical steps

- **Cleaning:** missing values, outliers, inconsistent categories.
- **Scaling:** standardization/normalization/robust scaling.
- **Encoding:** one-hot, ordinal, (careful) target encoding.
- **Dimensionality reduction:** PCA, feature selection.

Rule: Implement preprocessing inside a Pipeline and evaluate with CV.

Scaling & Normalization

Standardization (z-score):

$$x' = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

Min-max to [0, 1]:

$$x' = \frac{x - \min_{\text{train}}}{\max_{\text{train}} - \min_{\text{train}}}$$

Robust to outliers:

$$x' = \frac{x - \text{median}_{\text{train}}}{\text{IQR}_{\text{train}}}$$

Tips

- Tree-based models usually *don't* need scaling; linear/SVM/kNN/logistic *do*.
- **Fit** scalers on train folds only; **transform** val/test.
- Min-max can be replaced with some percentile from top and bottom - more robust.

code example

Missing Values & Outliers

Imputation (replacement of missing)

- Simple: mean/median/most-frequent; add *missingness indicator*.
- Model-based: kNN imputer, iterative (MICE). Fit on training fold only.

Outliers

- Prefer **robust scaling** or **winsorization** to blind removal.
- If removing, do it **inside** each CV training fold to avoid leakage.

Encodings

- **One-Hot** for nominal (watch high cardinality).
- **Ordinal** only if order is meaningful.
- **Target/mean encoding**: powerful but **must** be done with CV-inside-train (out-of-fold averages) to prevent leakage.

Recap Quiz I — Concepts & Splits

1. What is the difference between **empirical risk** and **expected (population) risk**, and which one do we ultimately care about?
2. Why must the **test set** be used exactly once? What goes wrong if you “peek”?
3. Explain the roles of **train**, **validation**, and **test** in one sentence each.
4. Give two examples of **data leakage**. How would you prevent each one?
5. When should you use **Stratified** vs. **Group-aware**?
6. In k -fold CV, what happens (pros/cons) when you **increase k** ?
7. Which models **need scaling** and which typically don't? Give examples.

Recap Quiz II — Metrics & Imbalance

1. When is **accuracy** misleading? Name two better metrics for rare positives.
2. **ROC AUC** vs. **PR AUC**: when do you prefer each, and why?
3. How to interpret **ROC AUC**?
4. Define **Precision**, **Recall**, and F_1 . What trade-off does the threshold control?
5. Give one **oversampling** and one **undersampling** technique. Where in CV do they belong?
6. For multiclass, explain **micro** vs. **macro** vs. **weighted** averaging.